MODULE 12 · DOMAIN 2 - APPLICATIONS AND INTEGRATION

Streaming, Batch and Files

Real time or later: streamed events in order, the Message Batches API at half price, and the Files API for documents you use again and again.

DOMAIN 2 of 8 EXAM WEIGHT 33.1% MODULE 12 of 36

admin@sthithapragna.com

A phone call or a letter? If someone is on the line, talk as you think. If nobody is, post it, at half the price.

A streamed response, event by event

01

Start

message_start: a Message with empty content.

02

Block start

content_block_start, one per text, tool use or thinking block.

03

Deltas

text_delta, input_json_delta, thinking_delta.

04

Block stop

content_block_stop: that block is complete.

05

Finish

message_delta with stop_reason and usage, then message_stop.

CURRENCY CHECK

ping events can arrive at any time, and new event types may be added. Ignore what you do not recognise.

Streaming with the SDK

```
1  with client.messages.stream(
2      model=TENANT_MODEL, max_tokens=2048,
3      system=SYSTEM_PROMPT, messages=history,
4  ) as stream:
5      for text in stream.text_stream:
6          relay_to_app(text)                  # each piece as it arrives
7      final = stream.get_final_message()
8
9  check_stop_reason(final.stop_reason)        # end_turn, max_tokens, refusal...
10 log_usage(final.usage)
```

5-6 The SDK assembles the deltas; you relay text as it comes.

7, 9 The final message still gets the module 3 checks.

THE TAKEAWAY

Use the SDK helpers rather than parsing raw events yourself, unless you have a reason not to.

Streaming details the exam likes

- **Tool input arrives as partial JSON.** Accumulate it, and parse when the block stops.
- **An error can arrive mid-stream,** after an HTTP 200, for example when overloaded.
- **Cancelling** means closing the stream: a stop button really stops generation.
- **The stop reason arrives at the end,** in message_delta. Check it before trusting the text.
- **Streaming improves time to first token** as people perceive it, not the total time.

> ON THE EXAM
>
> A 200 status means the stream started, not that it finished well.

When streaming is required

- **A person is waiting:** show the first words instead of a spinner.
- **Long requests:** "Consider using the streaming Messages API or Message Batches API for long-running requests, especially those over 10 minutes."
- **The SDKs refuse** non-streaming requests expected to exceed a 10-minute timeout.
- **Fast mode is different:** faster output, not a faster first token.

> THE TIEBREAKER
>
> Blank screen while waiting: stream. Total time too long: a faster model, fast mode or less output.

The Message Batches API

- **50% of standard prices.** Stacks with prompt caching.
- **Most batches finish within an hour;** any request not done in 24 hours expires.
- **Up to 100,000 requests or 256 MB** per batch; results kept for 29 days.
- **Results come back in any order:** match them with each request's custom_id.
- **Not supported in a batch:** streaming, fast mode. Not on Bedrock, Google Cloud or Foundry.

> ON THE EXAM
>
> Result types: succeeded, errored, canceled, expired. You are not billed for the last three.

A batch, from submit to results

```
batch = client.messages.batches.create(requests=[
    {"custom_id": f"tenancy-{t.id}",
     "params": {"model": SWEEP_MODEL, "max_tokens": 1024,
                "system": SWEEP_PROMPT, "messages": [note_request(t)]}}
    for t in tenancies_with_deadlines()          # code picks them, by date
])
# later, once batch.processing_status == "ended":
for r in client.messages.batches.results(batch.id):
    if r.result.type == "succeeded": save_note(r.custom_id, r.result.message)
    else: requeue(r.custom_id, r.result.type)    # errored or expired
```

2 A custom_id per request: the only safe way to match results.

9-10 Every result type is handled, not just success.

THE TAKEAWAY

Code decides which tenancies are due. Claude writes the notes. Rule one, again.

Real time or batch?

IF THE REQUIREMENT SAYS	CHOOSE
A person is waiting for the answer	Real time, streamed
Large volume, nobody waiting, cost matters	Message Batches API
Results needed by a fixed time	Batch early enough, plus a fallback
Needs fast mode or streaming	Real time
Each step depends on the last answer	Real time, or one batch per step

THE TIEBREAKER

"Within 24 hours" is a limit, not a schedule. Most batches finish within the hour, but plan for the limit.

The Files API

- **Upload once, reference by file_id** in later requests, real time or batch.
- **Up to 500 MB per file;** PDFs and text as documents, common formats as images.
- **Workspace-scoped:** "Never accept file_id values from end users or other untrusted sources."
- **Generally available** on the Claude API; not on Bedrock or Google Cloud; not ZDR-eligible.
- **Delete files** when the job is done, as Copperlet's document reader does.

> THE TIEBREAKER
>
> Files saves re-sending. Prompt caching saves re-processing. A repeated long document can use both.

Copperlet: two planted problems, fixed

QUIRK 6

Eight seconds of nothing

- Before: full response, then display
- After: streamed over the app's websocket
- Stop button closes the Claude stream
- Stop reason checked before saving

QUIRK 7

The overnight sweep

- Before: 310,000 synchronous calls at 2am
- After: code picks about 14,000 due tonight
- One batch submitted at 8pm
- Anything unfinished at 5am runs in real time

SETTLED

Quirk 6 settled; quirk 7's design settled. Module 13 handles the rate limits behind the 2am errors.

> A support chat waits for Claude's full response before showing it. Users see a blank panel for six to eight seconds on longer answers, and many leave.
>
> The total generation time is acceptable to the business.

What is the best fix?

A. Reduce max_tokens so that every answer is shorter

B. Send the questions through the Message Batches API

C. Stream the response and display the text as it arrives

D. Enable fast mode, so that the first words appear sooner

◆ PAUSE AND DECIDE

What is the best fix?

A Reduce max_tokens so that every answer is shorter

Truncates good answers. The wait is the problem.

B Send the questions through the Message Batches API

Built for nobody waiting. Someone is.

C Stream the response and display the text as it arrives

First words almost at once.

D Enable fast mode, so that the first words appear sooner

Fast mode speeds output, not the first token.

THE SIGNAL

"Total time is acceptable" plus a blank screen: the problem is perceived latency. Stream.

> An analytics team must classify 50,000 support tickets from the past quarter for a report due next week. No customer is waiting, and the budget is tight.
>
> Each ticket is classified independently.

Which approach fits best?

A Stream each classification, so the team can watch progress

B Submit them through the Message Batches API, with a custom_id per ticket

C Send synchronous requests from a hundred parallel workers

D Use fast mode on every request to finish sooner

◆ PAUSE AND DECIDE

Which approach fits best?

A Stream each classification, so the team can watch progress

Streaming serves people waiting. Nobody is.

B Submit them through the Message Batches API, with a custom_id per ticket

Half price, independent items, a week to spare.

C Send synchronous requests from a hundred parallel workers

Full price, and a rate-limit storm.

D Use fast mode on every request to finish sooner

Pays a premium for speed nobody needs.

THE SIGNAL

Independent items, nobody waiting, a tight budget, days to spare: Batch.

> A team submits a batch of personalised renewal letters, one request per customer. When processing results, they match each result to a customer by its position in the results file.
>
> Some letters are reaching the wrong customers.

What should they change?

A. Submit smaller batches, so that results come back in order

B. Sort the results by the time each one was created

C. Stream the batch results instead of downloading them

D. Match each result to its request by custom_id, never by position

◆ PAUSE AND DECIDE

What should they change?

A Submit smaller batches, so that results come back in order

Order is never guaranteed, at any size.

B Sort the results by the time each one was created

Creation time says nothing about which customer.

C Stream the batch results instead of downloading them

Changes how you read, not how you match.

D Match each result to its request by custom_id, never by position

Anthropic's documented way to match results.

THE SIGNAL

Anthropic: results "can be returned in any order". Always use custom_id.

> A streaming app has two bugs. It tries to parse a tool's input JSON on every input_json_delta, and throws errors. And when a stream that began with HTTP 200 ends with an error event, the app saves the partial answer as complete.
>
> The team wants to keep streaming.

Which two changes fix these bugs? Choose two.

A Accumulate the partial JSON and parse it when the block stops, or use the SDK helpers

B Move all tool calls to the Message Batches API

C Parse each partial JSON fragment with a forgiving regular expression

D Treat an error event as a failed request, discard the partial answer, and retry

E Ignore error events, since the response had already started successfully

◆ PAUSE AND DECIDE

Which two changes fix these bugs? Choose two.

A Accumulate the partial JSON and parse it when the block stops, or use the SDK helpers

Partial JSON is only valid when complete.

B Move all tool calls to the Message Batches API

Abandons streaming the team wants to keep.

C Parse each partial JSON fragment with a forgiving regular expression

Guesses at JSON that is not finished.

D Treat an error event as a failed request, discard the partial answer, and retry

A 200 means started, not finished well.

E Ignore error events, since the response had already started successfully

Exactly the bug.

THE SIGNAL

Two bugs, two fixes: parse partial JSON at block stop; treat mid-stream errors as failures.

> A nightly job submits a batch at 6am, and the results must be in a dashboard by 7am. Most nights it works, but on busy nights some results are not ready in time.
>
> The team wants to keep the batch discount.

What is the best fix?

A. Submit much earlier, and run anything unfinished near the deadline in real time

B. Turn on fast mode for the requests inside the batch

C. Set the batch requests to stream their results

D. Raise max_tokens so that each request completes in one pass

◆ PAUSE AND DECIDE

What is the best fix?

A Submit much earlier, and run anything unfinished near the deadline in real time

Plans for the limit, keeps the discount.

B Turn on fast mode for the requests inside the batch

Not supported in a batch.

C Set the batch requests to stream their results

Not supported in a batch either.

D Raise max_tokens so that each request completes in one pass

Unrelated to when a batch finishes.

THE SIGNAL

Most batches finish within the hour, but the limit is 24. A deadline needs slack and a fallback.

A contract assistant sends the same 300-page agreement, encoded in full, with every question a customer asks about it. Requests are slow to upload. Each customer's files must stay private to that customer.

The app runs on the Claude API.

What is the best change?

A Compress the encoded agreement more tightly before each request

B Paste the agreement's text into the system prompt on every request

C Upload it once with the Files API, keeping each customer's file_id on the server

D Let each customer send the file_id of the agreement they want to ask about

◆ PAUSE AND DECIDE

What is the best change?

A Compress the encoded agreement more tightly before each request

Still re-sends it every time.

B Paste the agreement's text into the system prompt on every request

Still re-sends it, and loses the PDF's layout.

C Upload it once with the Files API, keeping each customer's file_id on the server

Upload once, reference safely.

D Let each customer send the file_id of the agreement they want to ask about

Anthropic warns against exactly this.

THE SIGNAL

Repeated upload: Files API. Private per customer: the server owns file_ids, never the user.

Module 12 in five lines

- **Someone waiting?** Stream. Nobody waiting? Batch, at half price.
- **Events in order;** partial JSON parsed at block stop; errors can follow a 200.
- **Streaming fixes the first token;** fast mode fixes output speed.
- **Batch:** custom_id to match, any order, 24-hour limit, plan a fallback.
- **Files API:** upload once, server owns the file_id, delete when done.

COMING UP

Next: rate limits, errors, retries and third-party platforms.